Exercícios | Git & GitHub (Aula 3)

Aluno: Theo de Melo Xavier **RA: 22650114**

Parte 1 - Complete o comando

1. git **restore** <nome-do-arquivo>
2. git reset **--soft** HEAD~1
3. git reset **--hard** HEAD~1
4. git **revert** HEAD
5. git **reflog**
6. git restore **--source**=a1b2c3d -- index.html
7. git switch **-c** recuperado a1b2c3d
8. git commit **--amend** -m "nova mensagem"
9. git **rebase** main
10. git **clone** <url-do-seu-fork>

Parte 2 - Verdadeiro ou Falso

11. **V**
12. **V**
13. **F** - O git revert cria um novo commit que desfaz o anterior; o histórico é mantido.
14. **F** - Alterações nunca commitadas não ficam no reflog, então não dá para recuperá-las.
15. **V**
16. **F** - O --amend cria um novo commit, com hash diferente.
17. **V**

Parte 3 - Ordene o fluxo

1. Fork > sua cópia
2. Clone > baixa o repo
3. Branch > isola a mudança
4. Commit > salva o progresso
5. Push > envia ao GitHub
6. Pull request > pede o merge
7. Revisão > ajustes e aprovação
8. Merge > a mudança entra na main
9. Sync > atualiza o fork

Parte 4 - Perguntas curtas

19. Diferença entre git reset e git revert? Quando preferir o revert?

R: O reset move o HEAD (e a branch) para um commit anterior, descartando commits do histórico. O revert cria um novo commit que desfaz o efeito de outro, sem apagar nada. Prefiro o revert quando o commit já foi enviado ao GitHub ou o repositório é compartilhado, pois não reescreve o histórico.

20. Por que fazer fork em vez de push direto no repositório de outra pessoa?

R: Normalmente você não tem permissão de escrita no repositório alheio. O fork cria uma cópia na sua conta, onde você pode alterar livremente, e depois propõe as mudanças por Pull Request, que o dono revisa antes de aceitar.

21. Por que o --amend vira um problema depois do push?

R: Porque ele reescreve o commit (novo hash). O remoto e outras pessoas ainda têm o commit antigo, então o histórico diverge e seria preciso um push forçado (--force), que pode sobrescrever o trabalho de outros.